

CSCE 605 COMPILER DESIGN · MODULE 13 · LECTURE NOTES

Shader Compilation, Testing, and Shipping

The compiler you already depend on, and how to know yours is right.

MODULE OUTCOMES

- Explain the shader compilation pipeline end to end.
- Explain pipeline state objects and shader compilation stutter.
- Explain the shader permutation problem.
- Apply differential and randomised testing to a compiler.
- State what a compiler can honestly promise.

1 The shader compilation pipeline

```
HLSL / GLSL source
|  OFFLINE, at build time (DXC, glslang)
|  lex, parse, type check, optimise      (Modules 02-09)
v
SPIR-V or DXIL      <- an SSA IR (Module 07) you can disassemble
|                    and that ships inside your game package
|  AT RUNTIME, INSIDE THE GRAPHICS DRIVER
|  a WHOLE SECOND COMPILER: instruction selection,
|  scheduling, register allocation, for an ISA that is
|  undocumented and differs per GPU generation
v
GPU machine code

* the second compile runs on the USER'S machine
* it runs when a PIPELINE STATE OBJECT is created
* its register allocation decides OCCUPANCY (CSCE 735 M08)
* you cannot see, profile, or control it
```

Two compilers, and you control only the first. The one that performs register allocation — and therefore determines occupancy, and therefore determines whether your shader achieves its memory latency hiding (CSCE 735 Module 08 §3) — runs on the user's machine, is written by the GPU vendor, and changes with every driver release. **That is why shader performance can change without your code changing**, and why vendors ship game-specific driver optimisations.

WHY SHADER COMPILATION STUTTER HAPPENS

The driver compiles when a pipeline state object is created, not when the shader is loaded. **A PSO bundles the shaders together with the blend state, depth state, raster state, and render target formats**, because the driver specialises the generated code for all of them — fixed-function state that used to be hardware is now folded into the shader.

So a new material, a new effect, or merely a new combination of existing state means a fresh compile — tens to hundreds of milliseconds — **on the frame it is first needed**, which is typically the frame the player first walks into a new area or fires a new weapon.

That is the hitch. It is a compiler running inside your frame budget, and it is why the stutter correlates with new content rather than with load.

The three fixes are all now standard and all require deliberate engine work: precompile every PSO at load time so the wait becomes a loading screen or a shader-compilation progress bar; **ship a pipeline cache** so the work is done once per machine rather than once per run; or **compile on a background thread and substitute a placeholder material** until it is ready. **Explicit APIs made this visible rather than causing it** — older APIs did the same compilation, less predictably, which was arguably worse.

2 The permutation problem

SHADER PERMUTATIONS MULTIPLY

Shaders are specialised by preprocessor flags — the number of lights, whether shadows are enabled, normal mapping, skinning, fog, instancing, which lighting model, which tone curve.

n independent boolean features produce 2^n variants. Twenty features is a million shader programs, each of which must be compiled, stored, and shipped. Real engines have had hundreds of thousands.

Specialisation is exactly why they are fast. A variant with shadows disabled contains no shadow code at all — no dynamic branch, no unused registers holding shadow parameters, and therefore **higher occupancy** (CSCE 735 Module 08). The alternative, one shader branching at runtime, pays register pressure for code it does not execute.

So the trade is compile time and package size against runtime speed, and the standard responses are: **uber-shaders** with dynamic branching (fewer variants, lower peak performance), **on-demand compilation** of only the variants actually used, and **specialisation constants** — which SPIR-V supports directly, allowing one module to be specialised at pipeline creation time rather than at build time. **Specialisation constants are the principled answer**, and they move the combinatorial cost from your build to the user's PSO creation, which is §1's problem again.

3 Testing a compiler

DIFFERENTIAL TESTING FINDS WHAT TEST SUITES CANNOT

Generate a random program, compile it with two different compilers — or with the same compiler at two optimisation levels — run both, and compare the outputs.

Any difference is a bug in one of them, and crucially **no expected output is needed**. That is what makes the technique scale to millions of programs: the oracle is the comparison itself.

The generator must avoid undefined behaviour (Module 01 §4), or every difference is a false positive — two compilers may legitimately produce different results for a program with signed overflow in it. **This is the genuinely hard part of building one**, and most of Csmith's engineering is in guaranteeing that its generated programs have defined behaviour.

Csmith found over 400 previously unknown bugs in GCC and LLVM, and **the majority were in the optimiser** — which is exactly where hand-written test suites are weakest, because nobody writes a test for the interaction between three passes that only misbehaves when a loop is unrolled after being vectorised.

- **Differential testing against a reference implementation**, on generated programs. **The highest-yield technique available by a wide margin**, and the one to build first.
- **Self-differential testing: compare -O0 against -O2**. Requires no second compiler at all, and finds optimiser bugs specifically — which is where they are.
- **Metamorphic testing**: transform the source in a meaning-preserving way (rename variables, reorder independent statements, wrap in a function) and require the output to be unchanged. The same technique as CSCE 620 Module 13 §2, and equally cheap here.
- **An IR verifier run after every pass**, checking well-formedness and dominance (Module 07 §3). **This catches a broken pass immediately rather than three passes later**, when the symptom has become unrecognisable. **It is the cheapest item on this list and finds the most bugs per hour spent**.
- **Automatic test case reduction** — C-Reduce, llvm-reduce, or your own delta debugger. **A 3,000-line failing program is useless; the same bug reduced to 10 lines is an actionable bug report**, and reduction is mechanical enough to automate.

TRANSLATION VALIDATION, AND VERIFIED COMPILERS

Translation validation checks each individual compilation rather than the compiler in general: prove that *this* output is equivalent to *this* input. It is a weaker guarantee that is vastly easier to obtain. **Alive2 does this for LLVM optimisation passes and has found real, long-standing bugs**, including in transformations that had been in the compiler for years.

A verified compiler proves the whole thing once, mechanically. CompCert is a C compiler proved correct in the Coq proof assistant, with a machine-checked theorem that the generated assembly refines the source semantics.

Csmith found no miscompiles in CompCert's verified parts — bugs were found only in its unverified front end. **It was the only compiler tested of which that was true**, which is about as strong an empirical endorsement as a formal method has ever received.

The cost is enormous — person-decades of proof engineering — **and the generated code is slower than GCC's**, because every optimisation must be proved and the aggressive ones are hard to prove. **So it is used where a miscompile is unacceptable**: avionics, medical devices, nuclear control, and safety-critical automotive. **A real answer at a real price**, which is the correct way to think about formal verification generally.

4 Claiming honestly

- **'Conforms to this language subset, with these documented extensions and these known deviations.'** Every compiler has deviations; the honest ones list them.
- **'Tested differentially against GCC on ten million generated programs; three miscompiles found and fixed.' That is the claim** — the number of programs and the number of bugs — **not 'correct'**. It states the evidence and lets the reader judge the confidence.
- **'This optimisation level performs these transformations and assumes the absence of undefined behaviour.'** The second clause is the one users need and rarely get (Module 01 §4).
- **'Debug information is accurate at -O0 and degrades predictably above it.'** Rather than claiming it works, say how it fails (Module 10 §4).
- **And what you cannot honestly say: 'it generates correct code'. CompCert can say something close to that, with a machine-checked proof and a stated set of assumptions. You cannot**, and the gap between 'extensively tested' and 'proved' is exactly the gap that Module 01 §4 said makes compiler bugs so expensive.

WHERE THIS LEAVES YOU

You can build a compiler end to end — lexer, parser, resolver, type checker, SSA construction, dataflow analysis, optimisation, instruction selection, register allocation — **and you know which representation makes each phase possible**, which was the organising claim of Module 01.

And you understand the toolchain you were already depending on: what -O2 is actually doing, why undefined behaviour is dangerous rather than merely unspecified, why a debugger says 'optimised out', why your shaders stutter the first time an effect appears, and what the graphics driver is doing on the user's machine that you cannot see.

CSCE 614 described the machine this targets. CSCE 629 supplied the algorithms the optimiser applies. CSCE 735 explained what the generated code has to exploit to be fast, and CSCE 620 supplied the geometry the engine runs on.

CSCE 678 finishes the semester. Once the code is compiled and correct and fast, **the remaining question is how more than one machine runs it together** — which turns out to be a problem with its own impossibility results, its own failure modes, and its own reasons for being harder than it looks.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Khronos — SPIR-V specification, SPIRV-Cross, and SPIRV-Tools (free)	The §1 pipeline. spirv-dis and spirv-opt let you watch the offline half of it happen.
Yang, Chen, Eide & Regehr — Finding and Understanding Bugs in C Compilers (free)	Csmith. The method of §3, the results, and the CompCert comparison. The most important paper in this course.
Lee, Hur, Nagarakatte et al. — Alive2 (free)	Translation validation for LLVM passes , usable on your own transformations.
Leroy — CompCert, and Formal Verification of a Realistic Compiler (free)	The verified compiler of §3, with the papers explaining what exactly is proved and what is assumed.
Microsoft — DirectX Shader Compiler; and the Vulkan pipeline cache documentation (free)	The practical §1 and §2 material: PSO creation, pipeline caches, and specialisation constants.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Disassemble a shader to SPIR-V** and identify its basic blocks, ϕ instructions, and entry point.
- 2 Run spirv-opt with and without optimisation and diff the result.
- 3 **Measure PSO creation time** in a Vulkan or D3D12 application and report the distribution.
- 4 Implement a pipeline cache and measure the second-run improvement.
- 5 **Count the permutations** in a real shader system you have access to, and estimate the total compile time.
- 6 Convert one permutation axis to a specialisation constant and compare variant count and performance.
- 7 **Build a random program generator** for your language that avoids undefined behaviour.
- 8 **Differentially test your compiler against itself** at two optimisation levels on a million programs. Report what you find.
- 9 **Add an IR verifier and run it after every pass.** Report how many bugs it catches that your tests did not.
- 10 Implement delta-debugging reduction and reduce a failing case to under twenty lines.
- 11 **Project 2 is now due.** Submit the compiler, the per-pass measurements including negative results, the pass-ordering evidence, the differential testing results, and the honest comparison against gcc -O2.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the shader compilation pipeline and say where each compile happens.
- 2 Why does the driver's register allocator determine your performance?
- 3 Explain shader compilation stutter and give three fixes.
- 4 Why do permutations multiply, and why is specialisation worth it?
- 5 What are specialisation constants and what do they move?
- 6 Describe differential testing and say why the generator is the hard part.
- 7 Name five compiler testing techniques and say which is cheapest.
- 8 Contrast translation validation with a verified compiler.
- 9 Give four honest claims and one thing you cannot say.